CG 第一次作业报告

PB23000052 杨硕

一、问题背景

交互式绘图是计算机图形学中的基础应用之一。本次作业要求实现一个名为 MiniDraw 的交互式画图小程序，支持绘制多种基本图形图元，并通过面向对象程序设计的思想来组织代码结构。

1.1 任务要求

- 实现直线 (Line)、椭圆 (Ellipse)、矩形 (Rectangle)、多边形 (Polygon) 等图元的绘制；
- 每种图元需用一个类封装，各种图元从父类 Shape 继承；
- 使用类的多态调用绘图函数；
- 学习 GUI 编程（基于 Dear ImGui）、STL 的使用及 OOP 的封装、继承与多态。

1.2 开发环境

- 框架：Framework2D，基于 Dear ImGui 的 2D 图形界面库
- 渲染：OpenGL（通过 GLFW、GLAD 提供支持）
- 语言：C++20

二、算法原理与实现

2.1 整体架构

程序采用 MVC 风格的组织方式：`MiniDraw` 窗口负责 UI 按钮与布局，`Canvas` 负责画布绘制与鼠标事件处理，`Shape` 及其子类负责各图元的数据存储与绘制逻辑。所有图元通过 `std::vector<std::shared_ptr<Shape>>` 统一管理，利用多态在绘制时调用各自的 `draw()` 方法。

2.2 类继承关系

各形状类均继承自抽象基类 `Shape`，实现其纯虚函数 `draw()` 与 `update()`。类图如下：

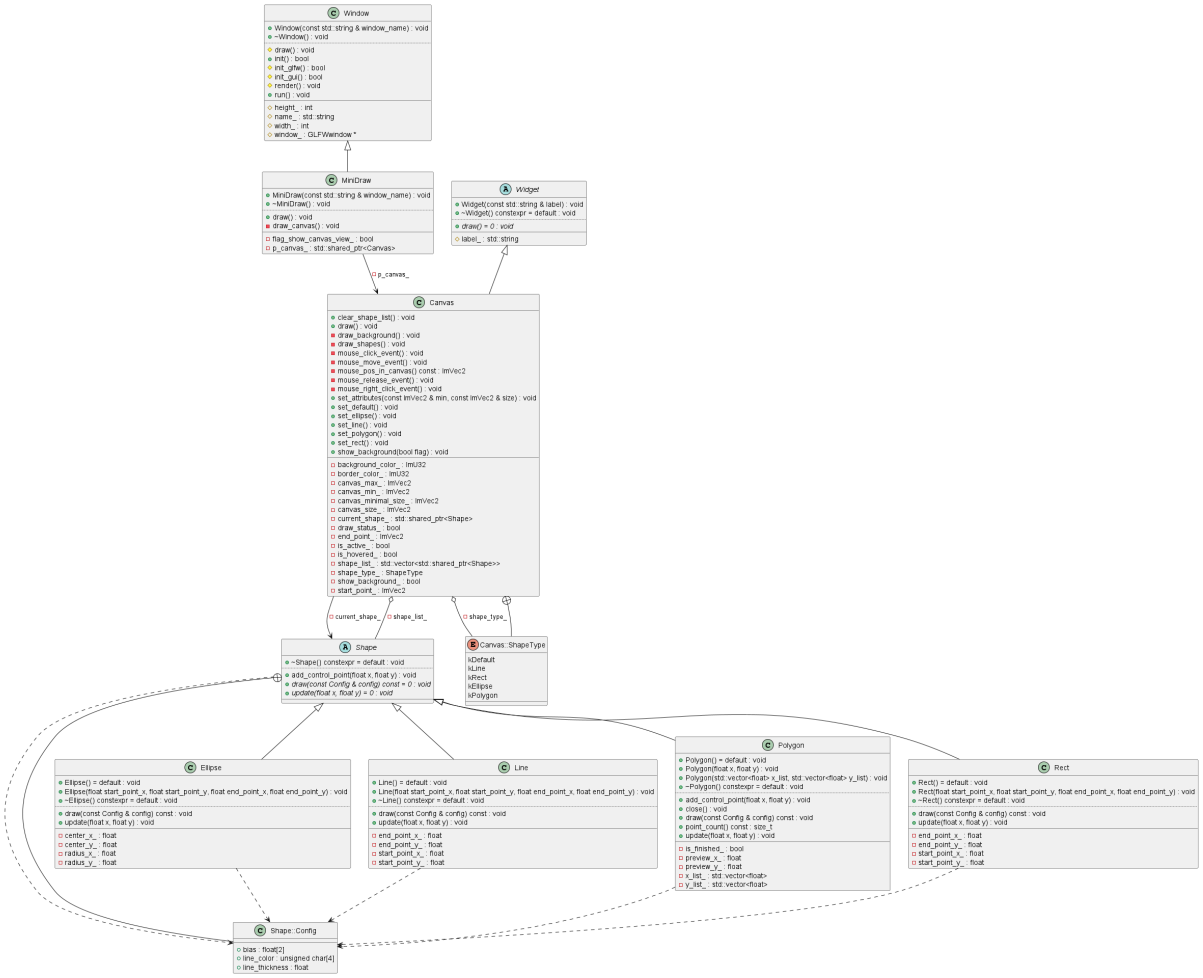

Figure 1: 1_MiniDraw 类关系图

2.3 Shape 基类

`Shape` 定义了统一的接口：

- `draw(const Config& config)`：纯虚函数，在画布上绘制图形
- `update(float x, float y)`：纯虚函数，根据鼠标位置动态更新图元
- `add_control_point(float x, float y)`：虚函数，用于多边形等需要多顶点的图元添加控制点

`Config` 结构体提供画布偏移、线条颜色和粗细等绘制参数。

2.4 各图元实现

2.4.1 Line 与 Rect（框架已有）

直线与矩形由起点和终点定义。`update()` 更新终点坐标，实现拖拽时实时预览。

2.4.2 Ellipse（椭圆）

椭圆采用“中心 + 半径”的表示：中心为第一次鼠标点击位置，半径为鼠标拖拽终点与中心的差值。

- 数据：`center_x_`，`center_y_`（中心），`radius_x_`，`radius_y_`（半径）

- 绘制：调用 ImGui 的 `AddEllipse(center, radius, col, rot, num_segments, thickness)`
- 交互：左键点击确定中心，拖动确定半径，再次左键完成

2.4.3 Polygon（多边形）

多边形用 `std::vector<float>` 分别存储顶点的 x、y 坐标。

- 数据：`x_list_`，`y_list_` 存储顶点序列；`preview_x_`，`preview_y_` 用于橡皮筋预览
- 绘制：依次连接相邻顶点成线段；绘制过程中从末顶点到当前鼠标位置画橡皮筋线；完成时调用 `close()` 连接首尾顶点
- 交互：左键添加顶点，右键结束绘制并闭合多边形

2.5 事件处理流程

Canvas 在每帧 `draw()` 中依次处理：

1. 绘制背景与 InvisibleButton 捕获鼠标
2. 左键点击 → `mouse_click_event()`：开始绘制或添加顶点（Polygon）/ 结束绘制（Line/Rect/Ellipse）
3. 右键点击 → `mouse_right_click_event()`：仅 Polygon 模式，结束绘制并闭合
4. 鼠标移动 → `mouse_move_event()`：更新 `end_point_` 并调用 `current_shape_->update()`
5. 绘制 `shape_list_` 与当前正在绘制的 `current_shape_`

三、实验结果与分析

3.1 功能验证

程序成功实现了以下功能：

- Line：两点确定直线，拖拽实时预览
- Rect：对角两点确定矩形
- Ellipse：中心点 + 半径确定椭圆，支持非正圆
- Polygon：多点确定多边形，橡皮筋预览，右键闭合

3.2 界面展示

运行 `1_MiniDraw.exe` 后，界面顶部提供 Line、Rect、Ellipse、Polygon 四个按钮，下方为全屏画布。用户选择图元类型后，在画布上通过鼠标完成绘制。

3.3 多态与封装

- 封装：每种图元的几何数据与绘制逻辑封装在各自类中，外部仅通过 `draw()` 与 `update()` 交互
- 继承：Line、Rect、Ellipse、Polygon 均继承 Shape，复用接口定义
- 多态：Canvas 统一持有 `std::shared_ptr<Shape>`，调用 `shape->draw(config)` 时根据实际类型执行对应的绘制实现

四、结论

本次作业完成了 MiniDraw 交互式画图程序，实现了 Line、Rect、Ellipse、Polygon 四种基本图元。通过面向对象设计，代码结构清晰、易于扩展。后续可在此基础上增加线条粗细与颜色、填充、图元选取与变换等拓展功能。